

RV-MalTrace 当前进展

35T 硬件追踪证据链与论文推进路线

主题： RISC-V 硬件辅助行为追踪

类型： 论文进展汇报

核心问题： 当前能做到什么、边界在哪里、下一步做什么

2026 年 5 月 29 日

汇报提纲

- 一 研究定位——把 trace tap 放进论文故事线
- 二 实验数据与结果——trace 格式样例、各流程能力、数据与 gate
- 三 证据边界——现在能说什么，哪些还不能说
- 四 下一阶段——从原型闭环推进到论文贡献

本次报告的重点不是罗列文件——而是把当前仓库整理成可审阅的科研进展、风险边界和下一步 gate。

Outline

① 研究定位

② 实验数据与结果

③ 证据边界

④ 下一阶段

这不是一个单纯的 RTL demo

当前仓库已经从“在 RISC-V/CVA6 或 LiteX/VexRiscv 上加一个 trace tap”推进到可复现实验证据链：事件格式、仿真 golden、35T 板级运行、行为恢复、解释接口和 claim boundary 都有对应文件与检查脚本。

对论文进展报告而言，最稳妥的表达是：RV-MalTrace 现在可以支撑一个 35T / LiteX / VexRiscv 硬件追踪辅助的恶意行为证据链原型。这不是成熟检测器，也不是 CVA6 板级结论，但已经足够作为下一步论文路线的工程底座。

一句话进展：当前原型已完成受控行为矩阵的板级 trace 证据闭环，并开始把真实恶意代码参考中的行为抽象迁移到安全可执行样例。

当前技术链路（图 1）

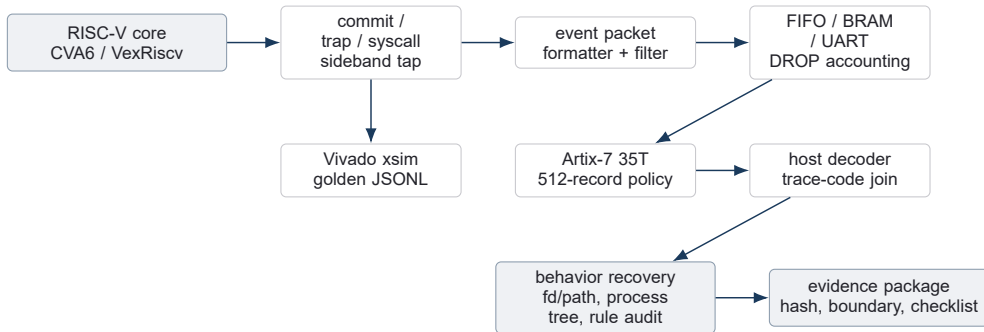


图 1 当前仓库覆盖从硬件事件采集到行为证据包的完整原型链路。

这条链路的关键点是低扰动事件采集和离线证据解释分离：硬件侧只保留提交事件与必要上下文，复杂语义在 host 侧恢复并受 gate 约束。

Outline

- ① 研究定位
- ② **实验数据与结果**
- ③ 证据边界
- ④ 下一阶段

仓库已经形成的四层能力（表 1）

层次	现在已经能做的事	主要证据
trace RTL / 格式	记录已提交、控制流、syscall、trap、context、ARG_MEM、DROP 与 MARKER，并有过滤、压缩原型、DROP 计数。	trace_format.md
Vivado / CVA6 仿真	trace unit、RVFI adapter、direct-core CVA6、full SoC probe、RV64GC microprobe 均有 PASS 记录。	sim_results.md
35T 板级实验	Artix-7 35T / LiteX / VexRiscv 下，13 个受控样例 full matrix 通过 512-record gate。	35T evidence bundle
行为解释与论文证据	支持 code map、trace-code join、fd/path、process tree、real-malware-derived lineage 和 claim boundary。	paper evidence chain

这四层已经让项目从“能跑一个样例”变成“能解释、能复现、能限制说法”的研究原型。

阶段实验数据总览（表 2）

流程/阶段	当前程度	实验数据	结果解释
Trace / Vivado	PASS	30 个 PASS 行；220 events；覆盖 syscall return、pointer、backpressure。	packet 语义、DROP 计数和 CVA6 仿真入口可复现。
35T 主矩阵	13/13 PASS	512 records；1,685 entry + 1,685 ret；130 marker；596 trap；max DROP 3.07%。	受控 benign + synthetic malware-like gate 闭环。
语义闭环	case-study PASS	fd/path 3/3 PASS，7 closed flows；process-chain 2 条边；function 6/6。	代表性解释路径可复述，不是完整语义恢复。
对比流程	8 PASS / 2 deferred	host/QEMU/strace/eBPF/QEMU-plugin/software/RVMT event-only 均有 13 样例证据。	baseline 框架已具备，pointer/helper 待补。
真实恶意派生	6/6 PASS	DarthRa/Mirai-derived；events 48–66；DROP 0；board 95.5–113.2 ms。	支持“参考行为可追踪”，不支持家族检测准确率。
资源/非干扰	CHECKED	LUT +48.04%，FF +4.97%，BRAM/DSP +0，slack 0.177 ns；6 项检查。	可引用实现成本和无反压原则，不能包装成性能提升。

Trace 事件模型已经稳定

当前事件模型围绕已提交行为设计，而不是完整 load/store 记录。syscall entry 从 U-mode ECALL 的 trap path 捕获，syscall return 从 SRET 返回 U-mode 捕获，DROP 作为一等事件进入 trace，避免在带宽不足时把 core 停住。

事件族	代表事件	用于回答的问题
控制流	RETIRE, BRANCH, JUMP	执行路径、压缩指令长度、跳转目标
syscall	SYSCALL_ENTRY, SYSCALL_RET	参数、返回值、持续周期、系统调用序列
异常与上下文	TRAP, CSR, SATP, PRIV	trap cause、地址空间、特权级切换
资源边界	ARG_MEM, DROP, MARKER	指针快照、溢出可见性、样例作用域

设计取舍：默认不追踪完整内存访问；需要语义时用 syscall-scoped ARG_MEM 或 side-channel 进行有界补全。

Trace JSONL 的具体格式 (示例 1)

35T 板级 trace 经 host converter 后输出为JSON Lines：每行一条事件记录。公共字段包括 record_index、cycle、evt、evt_code、pc、instr、priv、raw_header、raw_words；事件特有字段按需出现。

样例：benign/hello/board/trace-on/rep_00/trace.jsonl 的前三行解码结果。

#	evt(code)	cycle	pc / priv	该行实际 payload
0	MARKER(12)	19,457,332	0x00000000 / --	value=0xb0002cab; raw_header=0x0000000c; raw_words=16 个 32-bit word.
1	SYSCALL_ENTRY(4)	19,457,680	0x0101f554 / U	instr=0x00000073; syscall_id=0x0000b380; a0=1; a1=0x9d1c3ad8; a7=0.
2	SYSCALL_RET(5)	19,464,490	0xc068d6f4 / S	duration=6810; target=0x0101f558; a0=0; a7=0x193; raw_header=0x00000055.

可检查点：PPT 中的 trace 格式不是抽象引用；这里展示的是板级 artifact 中实际 JSONL 行的字段化内容。

16-word raw record 到 JSONL 的映射 (示例 2)

原始记录层保留为固定 16 个 32-bit word; JSONL 是在不丢弃 raw_words 的前提下, 把常用语义字段展开出来。下面同样来自 hello 的第 1 条 SYSCALL_ENTRY。

raw word	实际值	JSONL 字段含义
w0	0x00000004	raw_header; 低位解码为 evt_code=4, 即 SYSCALL_ENTRY。
w1	0x0128e690	cycle=19457680, 用于与 return、DROP、marker 做时序关联。
w2, w3	0x0101f554, 0x00000073	pc=0x000000000101f554; instr=ECALL。
w6	0x0000b380	syscall_id, 用于 entry/return 配对。
w8--w15	a0=0x00000001, a1=0x9d1c3ad8 a2=0x0, a3=0x1 a4=0x000f423f, a5=0x0 a6=0x0, a7=0x0	syscall 参数寄存器快照; 后续 fd/path、规则审计、baseline 对齐均基于这些字段。

格式结论: trace 文件采用“语义 JSONL + 原始 16-word 记录并存”的格式; 既能做实验统计, 也能回到 raw word 层复核转换。

Vivado 与 CVA6 仿真门禁 (表 3)

门禁	状态	说明
trace unit + RVFI adapter	PASS	覆盖 syscall return、pointer string、pointer guardrails、backpressure、filter、board minimal。
CVA6 direct-core matrix	PASS	smoke、branch、jump、ecall、illegal trap、ebreak 等 trace-on/no-trace 终态一致。
full SoC harness probes	PASS	breakpoint smoke、UART/MMIO store path、normal tohost/MMIO completion 已记录。
RV64GC microprobe	PASS	I/M/A/F/D/C 每类至少一条 committed instruction probe 通过。
Linux / arbitrary program	未声称	当前不是 Linux 全系统任意程序正确性结论，也不是 CVA6 物理板结论。

这部分适合在报告中说明：CVA6 仿真方向已有可复现基础，但论文级板上 Linux 语义仍要单独 gate。

仿真流程实验数据（表 4）

子流程	行数/状态	代表数据	实验结果
trace unit semantics	12 / PASS	pointer_string: 13 events / 5 ARG_MEM; pointer guardrails: 46 events / 14 ARG_MEM。	syscall pointer snapshot 的 synthetic 路径和保护条件通过。
overflow / filter	2 / PASS	backpressure: 30 events, 7 DROP records, 18 dropped events; filter: 仅保留 1 branch。	溢出可见且不反压 core，过滤逻辑可控。
RVFI adapter	1 / PASS	15 events, 覆盖 dual commit、U-mode entry/return、non-ECALL trap、compressed control flow。	CVA6 RVFI 到 trace schema 的适配已单测通过。
CVA6 direct-core	6 / PASS	smoke、branch、jump、ecall、illegal、ebreak; trace-on/no-trace final PASS 对齐。	真实 CVA6 committed RVFI 事件可进入同一 checker。
full SoC + RV64GC	9 / PASS	full SoC smoke/store/tohost + I/M/A/F/D/C microprobe; 30 行总计 220 events。	full SoC 最小探针与 RV64GC 扩展族最小 retire gate 已覆盖。

35T 主矩阵已经闭环 (表 5)

指标	结果	解释
run id	35t-smallcap-r512 full matrix 20260521	主 evidence bundle, 用于严格 trace gate。
样例矩阵	13/13 PASS	benign + synthetic malware-like, 全矩阵 gate 通过。
trace 容量	512 records	并非通过扩大到 1024 records 解决; 当前按 small-capacity policy 分样例配置。
trace health	UNKNOWN = 0	marker scope、runtime process attribution、DROP/capacity 均进入 gate; corrupt event 也为 0。
语义补全	selected PASS	targeted validation 关闭 fd/path 与 process-tree 代表性解释路径。

这里最有价值的是 “35T 小容量约束下仍可建立证据链”, 不是成熟恶意软件检测能力。

35T 样例矩阵实验数据（表 6）

样例组	样例数	事件计数	DROP	结果
benign 基线	5	545 entry / 545 ret / 50 marker	max 0	hello, ls, cat, cp, sha256sum 全部 PASS; ls 的 file-scan overlap 被标注为 benign overlap。
synthetic malware-like	8	1,140 entry / 1,140 ret / 80 marker / 596 trap	max 3.07%	8/8 PASS; illegal_trap 使用 p0c, 其余使用 p0a。
capacity-sensitive case	1	process_chain: 380 entry / 380 ret	0	不再是 512-record blocker; 每个 rep 154 events, 无 cap hit。
trap-heavy case	1	illegal_trap: 90 entry / 90 ret / 596 trap	3.07%	trap profile 捕获 illegal-instruction 行为, 仍低于 5% gate。
aggregate gate	13	3,370 syscall records + 130 marker + 596 trap	max 3.07%	13/13 strict sample gate PASS, UNKNOWN/corrupt 为 0。

illegal_trap 的板级结果样例（示例 3）

trace.jsonl 片段

trace-code join 摘要

#	事件	实际字段	字段	实际结果
0	MARKER	value=0xb0000f79; sample begin.	events	158
1	SYSCALL_ENTRY	pc=0x0101f554; syscall_id=0x0000e101; a0=1.	marker scope	PASS; begin 0xb0000f79, end 0xe0000f79
2	SYSCALL_RET	duration=6819; target=0x0101f558; a0=0.	runtime path	/usr/bin/illegal_trap; runtime map PASS
5	TRAP	cause=0x9; tval=0x73; pc=0xc000788c; priv=S.	PC owner	kernel 121, target 27, unknown 10
6	TRAP	cause=0x2; tval=0x60059593; supervisor trap.	callsite kind	illegal-instruction site 2; syscall site 14
13	TRAP	cause=0xc; pc=0x0001059c; priv=U.	DROP	max 3.07%, below 5% gate

判定顺序：先看到 trap-heavy trace、marker 闭合、runtime/process 归因与 code-map 结果，再给出结论：该样例在 512-record policy 下通过板级 gate。

真实恶意代码派生行为有六行证据（表 7）

行为行	来源	状态	边界
ELF header probe	DarthRa reference	PASS	真实恶意代码派生行为，不是 payload 等价执行。
rootkit device probe	DarthRa reference	PASS	安全控制下的行为验证。
virus fixture walk	DarthRa reference	PASS	文件遍历类行为抽象。
proc scan	Mirai reference	PASS	非网络版本，排除外联。
watchdog probe	Mirai reference	PASS	非网络行为抽象。
encoded table	Mirai reference	PASS	字符串/表行为抽象。

可讲法：这些结果支持“选取自真实恶意代码参考的行为可以被 35T 原型 trace、验证和规则审计”，不支持“真实家族检测准确率”。

真实恶意派生流程的时序与 trace 数据（表 8）

样例	Host ms	QEMU ms	Board ms	Events	DROP	状态
darthra_elf_header_probe	17.3	27.7	95.5	54	0	PASS
darthra_rootkit_device_probe	17.7	28.8	109.7	52	0	PASS
darthra_virus_fixture_walk	29.4	73.5	113.2	66	0	PASS
mirai_proc_scan	12.4	20.8	105.2	56	0	PASS
mirai_watchdog_probe	12.2	19.1	97.8	48	0	PASS
mirai_encoded_table	18.5	44.1	100.5	50	0	PASS

这些数据用于说明同一组安全控制行为在 host、QEMU 和 35T trace-on 三条路径下均有对应记录；比值是描述性结果，不是性能等价或加速结论。

解释接口让结果可复述

现在的结果不是只能看原始 UART 或 JSONL。仓库 目前解释层已经覆盖三类代表性语义：已经提供终端解释接口，把 trace health、gate 状态、恢复出的行为、可疑 cue、证据来源和 non-claim 一起打印。

命令	用途
rvmt explain:35t	单样例解释
--flow	run-level 过程视图
--format markdown	生成可提交材料

1. **code map**——PC 与 ELF symbol / syscall site 关联；
2. **fd/path**——代表性 openat -> fd -> read/write/close 闭环；
3. **process tree**——clone 返回 child PID 与 wait PID 匹配。

语义恢复流程的实验结果（表 9）

流程	实验数据	状态	结果边界
fd/path flow	3/3 required samples PASS; file_scan=1 closed flow, batch=4, self_copy=2; unresolved fd = 0。	PASS	path string 来源是 board syscall side-channel, 不是硬件 pointer snapshot。
process tree	process_chain: 10 candidates, 5 PASS candidates; 2 条 clone-return/waitid 边; 2 个 /bin/true exec path。	PASS	parent PID 仍为 target_parent_unresolved, 不声 称完整 OS process graph。
function attribution	6/6 case-study samples available; ELF symbol table; 约 1,099–1,103 functions/sample; 119 target-attributed events。	PASS	function-level 是 symbol/range 证据, 不等于 source-line。
source-line attribution	6 个 case-study samples 均记 录 source_line_count=0; source line basis unavailable。	PARTIAL	下一步需要保留 DWARF/source-location records。

file_scan 的 fd/path 输出样例 (示例 4)

下面不是“fd/path PASS”的文字概括，而是 canonical summary 选中的 board syscall side-channel 结果。它展示了路径、fd 生成、目录读取和 close 如何闭合成一个 flow。

seq	name	fd	return	结果字段
12	openat	3	0x3	path=experiments/linux_behavior/malware_like/fixtures/scan_root; path_source=board_syscall_side_channel。
13	getdents64	3	0x70	目录项读取返回非零长度。
14	getdents64	3	0x0	第二次读取到目录末尾。
15	close	3	0x0	fd 生命周期闭合; fd_generation=1。

实验结果：openat(path) -> fd=3 -> getdents64 -> getdents64 -> close 被恢复为 status=closed; unresolved_fds=0，因此 fd/path flow 在该样例上判定为 PASS。

process_chain 的 process-tree 输出样例 (示例 5)

process-tree 结果来自同一类 canonical summary: 选取最强 side-channel candidate, 显式输出 clone 返回值、wait 目标 PID、exec path 和最终边。

证据项	实际输出	解释
observed counts	clone/fork = 2 execve = 4 wait = 2	目标样例中观察到两条进程创建链。
clone seq 12	return = 0xcb child PID = 203	parent-side clone return 被解释为子进程 PID。
wait seq 13	wait PID = 203 return = 0x0	wait 目标 PID 与 clone 返回 PID 匹配。
clone seq 14	return = 0xcc child PID = 204	第二条子进程边。
wait seq 15	wait PID = 204 return = 0x0	第二条 wait 匹配闭合。
exec path	/bin/true source=board syscall side-channel	exec path 来源明确, 不是文件名猜测。
edges	parent unres. → 203 parent unres. → 204 confidence: strong	父进程身份按边界保守输出为 unresolved, 不强行补全 parent PID。

实验结果: summary status 为 PASS, partial edges 与 unclosed edges 均为空; 但 parent PID 仍为 unresolved, 不声称完整 OS process graph。

对比实验流程当前数据（表 10）

baseline / ablation	状态	样例覆盖	当前结果
host native / host strace	PASS	13/13	host strace/native median = 2.60；用于说明软件 syscall 路径扰动。
QEMU native / QEMU strace	PASS	13/13	qemu strace/native median = 1.84；提供模拟器基线。
ebPF-only	PASS	13/13	bpfftrace raw syscall tracepoint, comm-filtered host binaries。
QEMU plugin	PASS	13/13	QEMU user-mode TCG plugin syscall-count baseline；不是硬件 trace。
software instrumentation	PASS	13/13	-finstrument-functions host binary；不提供 syscall 参数恢复。
RVMT event-only	PASS	13/13	trace bytes/syscall median = 672.9；board trace on/off median = 0.566，仅作描述。
pointer snapshot / helper	DEFERRED	0/13	pointer snapshot 与 helper/ebPF companion 仍是后续增强路径。

Outline

- ① 研究定位
- ② 实验数据与结果
- ③ 证据边界**
- ④ 下一阶段

现在可以稳妥陈述的结论（表 11）

可陈述结论	精确含义	证据位置
35T 原型闭环	Artix-7 35T / LiteX / VexRiscv 上，受控样例的硬件 trace、运行时归因、规则审计已闭环。	application closure
主矩阵 gate 通过	512-record policy 下，13 个样例 13/13 PASS，UNKNOWN/corrupt 为 0。	full matrix bundle
代表性语义闭环	fd/path 与 process-tree 代表路径已在 targeted validation 中 PASS。	semantic summaries
真实恶意派生行为可追踪 证据链纪律可用	DarthRa/Mirai reference 的 6 行安全控制行为验证 PASS。 hash manifest、claim boundary、review checklist 已形成 CCF-A-style discipline。	lineage / baseline strong chain package

这些说法的共同特点是：限定平台、限定样例、限定安全控制、限定证据类型。

仍然不能越界的说法（表 12）

不能声称	状态	原因
CVA6 物理板验证完成	NO	当前 35T 结果来自 LiteX/VexRiscv, CVA6 主要是仿真和综合/资源路径。
真实恶意软件检测准确率	NO	真实恶意派生行为不是家族分类、IOC/TTP 覆盖或检测质量评估。
完整语义恢复	NO	fd/path 与 process-tree 是代表性闭环; source-line、全局进程所有权、完整内存语义仍缺。
成熟检测器或产品级系统	NO	当前是 evidence-chain prototype, 规则审计是应用展示, 不是成熟 detector。
单 trace side-channel 全门 禁 PASS	NO	targeted validation 是 dual-channel; side-channel 不是 strict full-matrix trace gate。

建议表述：先讲已闭环证据，再主动讲 non-claims。这样能清楚呈现项目是“有边界的进展”，不是夸大。

资源与非干扰证据的边界 (表 13)

指标	Baseline	Trace-enabled	Delta
LUT	84,923	125,717	+40,794 (+48.04%)
FF	56,491	59,301	+2,810 (+4.97%)
BRAM18 equiv	108	108	+0
DSP	27	27	+0
Slack	0.177 ns	0.177 ns	0.000 ns

非干扰 gate 已检查 sideband-only、registered snapshot、DROP-not-stall、direct-core trace/no-trace final PASS 等条件。资源结果可以引用为当前 routed report 的 trace-enabled delta。

边界：这些数据不能被包装成 IPC/Fmax 提升或真实 workload overhead 结论；它们说明的是当前实现成本和非反压原则。

Outline

- ① 研究定位
- ② 实验数据与结果
- ③ 证据边界
- ④ 下一阶段

论文主线要从 prototype 走向 contribution

如果目标只是“硬件能采 syscall event”，研究贡献会偏工程。更有论文竞争力的主线应当是：RISC-V 低扰动硬件辅助 syscall 语义追踪，其中硬件 trace 提供 committed、低可见度的行为事实，离线语义层恢复 fd/path/process/memory-permission 等行为图。

下一阶段的中心问题不是再堆更多样例，而是回答：仅靠寄存器与上下文能恢复到哪里；哪些 pointer 参数必须补全；硬件 pointer snapshot、kernel helper/eBPF fallback 与 baseline 方法的边界分别是什么。

论文故事线：从“35T 原型可行”推进到“RISC-V hardware-assisted semantic behavior tracing with bounded pointer reconstruction and evasion-aware evaluation”。

下一步一：把 pointer 语义做成核心创新

许多关键 syscall 的行为语义不在寄存器数值本身，而在 pointer 指向的用户内存中。当前 ARG_MEM 已有 synthetic openat pathname 与 guardrails 回归，下一步要把它推进到 Linux/CVA6 信号接入或明确 fallback。

$$\text{SYSCALL_ENTRY}(a_7, a_0, \dots, a_5) + \text{ARG_MEM}(ptr, bytes) \\ \Rightarrow \text{openat}(\text{path}), \text{execve}(\text{argv}), \text{connect}(\text{sockaddr})$$

路径	要验证的点	风险
hardware snapshot	S-mode copy-from-user load 能否稳定捕获字符串/结构体片段。	LSU 信号、跨页、带宽
kernel helper/eBPF	只补 pid/path/string，不替代硬件 trace。	威胁模型收窄
offline reconstruction	fd/path/process/memory behavior graph 形成可审计输出。	完整性边界

下一步二：把对比实验补齐（表 14）

baseline	比较目标	关键指标
strace/ptrace	最直接的软件 syscall ground truth 与 anti-debug 可见性。	overhead, detectability
eBPF-only / helper	OS helper 路线的语义能力与扰动。	semantic accuracy, overhead
QEMU / plugin	模拟器路径的可控性与真实执行差距。	timing, coverage
software instrumentation	编译期或二进制插桩与硬件旁路采集的对比。	perturbation, completeness
RV-MalTrace ablation	event-only、pointer snapshot、helper 三种模式拆开比较。	accuracy, resource, drop

对系统安全论文而言，实验不能只证明“能 trace”，还要证明为什么硬件辅助比常规软件方法有价值。

未来两周优先级（表 15）

任务	交付物	判断标准
source-line attribution	在 code map 中保留 DWARF/source line，并 join 到代表 syscall/trap site。	source summary 从 PARTIAL 推进
Linux semantic gate	选择 2–3 个 open/exec/mmap/process workload，固定 runbook 与 ground truth。	path/process 语义可复查
pointer snapshot 决策	CVA6 LSU 信号 map + synthetic-to-Linux feasibility note。	hardware/fallback 路线明确
baseline plan	strace, QEMU, eBPF/helper 的可运行脚本和指标表。	同一样例可横向比较
paper claim table	将 allowed claims / forbidden claims 变成论文式 contribution boundary。	可直接审阅

优先级原则：先补“论文里一定会被问”的证据缺口，再扩样例规模。

未闭合流程的实验状态 (表 16)

流程	当前数据	状态	下一步 gate
source-line attribution	6 个 case-study samples: source line count = 0; function-level 6/6。	PARTIAL	保留 DWARF, 输出 source-location records。
hardware pointer snapshot	pointer snapshot samples_with_evidence = 0/13; synthetic ARG_MEM 仿真 PASS。	DEFERRED	CVA6 LSU map + Linux copy-from-user workload。
helper/eBPF companion	helper/eBPF companion evidence = 0/13; eBPF-only baseline 13/13 PASS。	DEFERRED	明确 trusted-kernel 边界, 只补语义。
side-channel single-run gate	sample status 13/13 PASS; strict sample gate 9/13 PASS。	not claimed	不能写成 single-trace all-gates PASS。
fuzz/stress validation	fuzz_cf、fuzz_trap、fuzz_syscall、fuzz_context、fuzz_overflow: 5 TODO。	TODO	deterministic seeds + invariant checker。

需要讨论的三个决策

接下来需要尽早确定方向，否则后续实验容易发散：

1. **论文定位**——主打 malware analysis，还是更稳地表述为 RISC-V semantic behavior tracing，并把 malware 作为应用场景；
2. **语义补全路线**——优先冲 hardware pointer snapshot，还是 hardware trace + trusted helper 双路径并行；
3. **目标强度**——当前 35T evidence chain 是否作为阶段性小论文/技术报告，还是继续补 CVA6/Linux/对比实验后冲更高层级会议。

建议路线：短期保守声称 35T evidence-chain prototype；中期把 pointer semantic reconstruction 与 evasion-aware baseline 做成论文核心贡献。

请老师批评指正

RV-MalTrace 论文进展汇报 | 2026 年 5 月 29 日